

Project Executable Files

Date	15th July 2026
Team ID	xxxxxx
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	No
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA

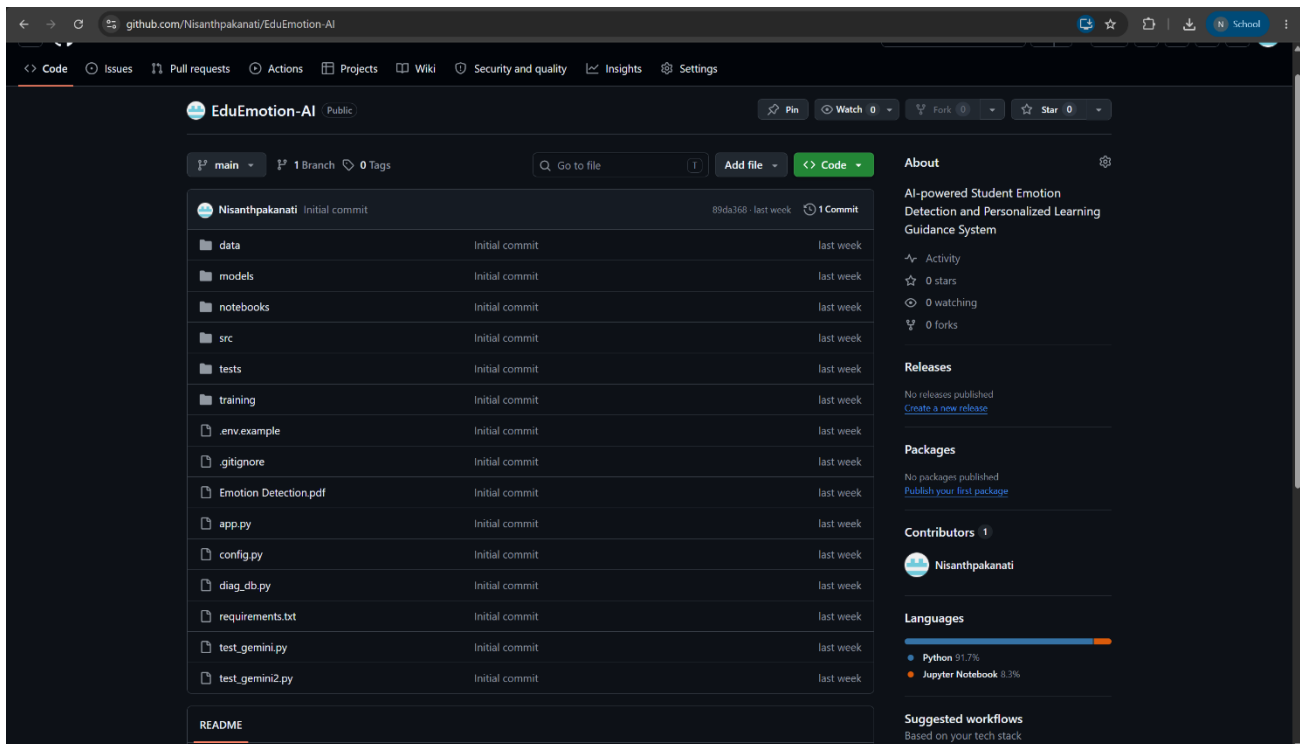
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

[Paste or describe your folder structure here. Example:]

```
/project-root
/src    - Source code files
/tests  - Test files
/docs   - Documentation
/config - Configuration files
README.md - Setup and run instructions
requirements.txt - Python dependencies (or package.json for Node)
```



The screenshot displays the GitHub interface for the repository 'EduEmotion-AI' by user 'Nisanthpakanati'. The repository is public and has 1 commit. The file structure is as follows:

- data
- models
- notebooks
- src
- tests
- training
- .env.example
- .gitignore
- Emotion Detection.pdf
- app.py
- config.py
- diag_db.py
- requirements.txt
- test_gemini.py
- test_gemini2.py
- README

The right sidebar provides additional information:

- About:** AI-powered Student Emotion Detection and Personalized Learning Guidance System. Activity: 0 stars, 0 watching, 0 forks.
- Releases:** No releases published. [Create a new release](#).
- Packages:** No packages published. [Publish your first package](#).
- Contributors:** 1 contributor: Nisanthpakanati.
- Languages:** Python 91.7%, Jupyter Notebook 8.3%.
- Suggested workflows:** Based on your tech stack.

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	NA
Login Credentials (Demo)	nishanthmanikanta@gmail.com , nish123
Platform / Hosting Provider	Local Machine (Streamlit)
Repository Link	https://github.com/Nisanthpakanati/EduEmotion-AI
Demo Video Link	

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites

- Python **3.12+** installed
- Git installed
- Minimum **8 GB RAM** (recommended for BERT training)
- Internet connection (required for first-time DistilBERT download during training)
- Google Gemini API Key (for AI chatbot features)

Step 1: Clone the repository

```
git clone https://github.com/Nisanthpakanati/EduEmotion-AI.git  
cd EduEmotion-AI
```

Step 2: Create and activate a Python virtual environment

```
python -m venv venv
```

On Windows

```
venv\Scripts\activate
```

On Mac/Linux

```
source venv/bin/activate
```

Step 3: Install all required dependencies

```
pip install -r requirements.txt
```

If required, install additional libraries:

```
pip install torch transformers datasets accelerate sentencepiece  
safetensors
```

Step 4: Train the Machine Learning Models (First Time Only)

Train the BiLSTM model

```
python training/train_bilstm.py
```

Train the DistilBERT model

```
python training/train_bert.py
```

After successful training, the following files will be generated:

models/bilstm/

```
model.pt  
vocab.json
```

models/bert/

```
config.json  
model.safetensors  
tokenizer.json  
tokenizer_config.json  
special_tokens_map.json  
vocab.txt
```

These model files are required for emotion prediction and deployment.

Step 5: Configure Environment Variables

Create a `.env` file in the project root and add:

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

Step 6: Launch the Streamlit Application

```
streamlit run app.py
```

Step 7: Access the Application

Open your browser and visit:

<http://localhost:8501>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	BERT model requires initial training before first deployment.	Train locally and upload generated model files.
2	Gemini AI features require a valid Google Gemini API key.	Add the API key in the .env file.
3	CPU-based BERT training is slow on systems without a GPU.	Use a GPU if available or wait for training to complete.